

WEEK 08 · JAVASCRIPT · THE AI WORKFLOW

# THE LANGUAGE YOU ALREADY KNOW

JavaScript for people who already program — and the method you will use for the rest of the course.

---

## WHERE WE ARE

Six weeks of pages that render and work. Today they start to think.

---

## AFTER TODAY YOU CAN

Read JavaScript that surprises you, and say why it did.

---

## WHAT THIS UNLOCKS

Your three pages, and the first spec you write before the code.

---

*“Java and JavaScript are as similar as car and carpet.”*

THE OLD COURSE’S JAVASCRIPT LESSON, KEPT — STILL THE MOST USEFUL SENTENCE IN THIS DECK

TARGET · MINUTE 0

# THIS IS WHERE YOU WILL BE AT MINUTE 180

the bench of class assignment 8, every row green – four pure functions and one repaired review

reload

open

## The bench

Parts 1 and 2. Save a file, refresh this page. Green here is the floor, not the grade.

### FILTERBYQUERY

5 of 5 passing

finds one, ignoring case

an empty query returns everything

spaces around the query are ignored

Hebrew works too

it does not touch the array it was given

### SORTITEMS

4 of 4 passing

### The bench

Parts 1 and 2. Save a file, refresh this page. Green here is the floor, not the grade.

#### FILTERBYQUERY

5 of 5 passing

finds one, ignoring case

an empty query returns everything

spaces around the query are ignored

Hebrew works too

it does not touch the array it was given

#### SORTITEMS

4 of 4 passing

by title, and returns to default property

by year, added first, not year last

by rating, then reviews by title

one property, is NOT recommended

targets/week-08.png · 1280

סיכום פריט [הרשימה](#) מדף התקליטים

## המדף

התקליטים שאספתי, עם דירוג ושנה.

375 · the list page, toolbar outside the list

► **THE POINT** — four things leave this room: fourteen predictions with the gap explained, four **PURE** functions that pass this bench without touching the array they are handed, one review of code Claude wrote, and your first `FEATURE_SPEC.md` with three pages built from it. You do each once in a cycle first.

## AI THIS WEEK

# PARTNER. IT WRITES THE CODE; YOU WROTE THE SPEC, AND YOU CHECK WHAT CAME BACK.

LAYER · WEEKS 8–12 · PARTNER

## ALLOWED

Claude implements a feature **from your spec**: the filter and sort in cycles 4–5, the three pages of part 4 from your `FEATURE_SPEC.md`. Asking it to review — as a claim, checked against the bench.

## NOT ALLOWED

Code with no spec. A commit with no review line. And parts 1–3 of the assignment — predict, the four functions, the review — **you read and write those alone**, because week 13 does.

## NO SUBSCRIPTION?

Everything works in the free chat, and nothing in the course depends on a paid plan. Paste the spec, copy the file back by hand, run the same bench. Graded identically.

# WHAT TODAY IS, AND IS NOT

## NOT TODAY

Variables. Loops. `if`. Functions as an idea. Arrays as an idea. You have all of it.

## TODAY

The seven places where what you know is **wrong here**, and the idioms your project is built from.

## AFTER THE BREAK

The method: a spec to decide, Claude to write, a bench to check, and a review that is graded.

► **SAY THE DEAL OUT LOUD** — if any of this is genuinely new rather than merely different, `primer/` covers it, with solutions, and nobody is graded on it. Hoisting, the temporal dead zone, closures and the event loop are in the reference deck, not in a cycle — week 10 and week 11 use them.

CYCLE 1 · 25 MIN

# COERCION AND TRUTH: WHAT THE FORM GAVE ME

A form field gives you a string, or nothing at all. One function, `stockLabel(row)`, turns what arrived into a label — and on the way: the `==` triangle, why `===`, the one exception (`== null`), `Number('')` that is zero, and `??` that keeps a zero a zero. `app.js` is a module: Live Server, and `import` with `./` and `.js`.

## 1 I DO

10 MIN

3 on the next three slides, 7 in the editor · live-code/01-coercion-and-truth · the triangle · if (`row == 0`) (wrong) · trim, `Number`, `Number.isNaN`, `===` · (`row ?? ''`) · copies `||` / `??`

## 2 WE DO

5 MIN

you type steps 1, 3, 4 · compare with checkpoint 1 · fourteen rows, "not entered" twice

## 3 YOU DO

8 MIN

`priceLabel(row): "no price" · "free" · "12.5" · "not a number" — and no == inside it`

## 4 DEBRIEF

2 MIN

`Number(row)` before the empty check makes the empty field "free" · ↗ reference

## SIX LINES. EVERY ONE OF THEM IS LEGAL.

```
1 [] + []           // ""
2 [] + {}           // "[object Object]"
3 0.1 + 0.2         // 0.30000000000000004
4 [10, 9, 1].sort() // [1, 10, 9]
5 typeof null       // "object"
6 '5' - 3           // 2      ... but '5' + 3 is "53"
```

You will not need **any** of these lines. You will need to not be surprised by them.

# THE TABLE NOBODY MEMORISES, COMPUTED LIVE

every cell is a real `a == b`, and it explains itself

↺ reload

open ↗

LOOSE EQUALITY, EVALUATED BY THIS PAGE

☒ compare with == ☐ compare with === ☐ highlight the broken triangle

Click a cell to see how the specification got there.

| ==        | 0     | 1     | '0'   | ''    | 'false' | false | true  | null  | undefined | NaN   | []    | [0]   |
|-----------|-------|-------|-------|-------|---------|-------|-------|-------|-----------|-------|-------|-------|
| 0         | true  | false | true  | true  | false   | true  | false | false | false     | false | true  | true  |
| 1         | false | true  | false | false | false   | false | true  | false | false     | false | false | false |
| '0'       | true  | false | true  | false | false   | true  | false | false | false     | false | false | true  |
| ''        | true  | false | false | true  | false   | true  | false | false | false     | false | true  | false |
| 'false'   | false | false | false | false | true    | false | false | false | false     | false | false | false |
| false     | true  | false | true  | true  | false   | true  | false | false | false     | false | true  | true  |
| true      | false | true  | false | false | false   | false | true  | false | false     | false | false | false |
| null      | false | false | false | false | false   | false | false | true  | true      | false | false | false |
| undefined | false | false | false | false | false   | false | false | true  | true      | false | false | false |
| NaN       | false | false | false | false | false   | false | false | false | false     | false | false | false |
| []        | true  | false | false | true  | false   | true  | false | false | false     | false | false | false |
| [0]       | true  | false | true  | false | false   | true  | false | false | false     | false | false | false |

35 of 144 pairs are equal under ==. The diagonal accounts for 9 of them. Everything else is a conversion you did not ask for.

HOW IT GOT THERE

Pick a cell.

► ASK FOR `'' == 0`, THEN `0 == '0'`, THEN `'' == '0'` — and wait. Only then switch on the broken triangle.

# THE RULE, AND THE ONE EXCEPTION WORTH KEEPING

## NEVER

```
1 if (value == 0)
2 if (input == false)
3 // "" passes. [] passes.
```

## ALWAYS

```
1 if (value === 0)
2 if (input === false)
```

## THE EXCEPTION

```
1 if (value == null)
2 // null AND undefined,
3 // and nothing else
```

**===** asks one question. **==** asks a question and then negotiates.

## CHECKPOINT 1

## YOUR SCREEN SHOULD LOOK LIKE THIS

Week 8 · cycle 1 · coercion and truth

## What the form gave me

One row for every print() in app.js, in the order they ran.

| what app.js printed              | the value      |
|----------------------------------|----------------|
| <code>'' == 0</code>             | true           |
| <code>0 == '0'</code>            | true           |
| <code>'' == '0'</code>           | false          |
| <code>'' === 0</code>            | false          |
| <code>Number('')</code>          | 0              |
| <code>stockLabel("3")</code>     | "3 in stock"   |
| <code>stockLabel("0")</code>     | "sold out"     |
| <code>stockLabel("")</code>      | "not entered"  |
| <code>stockLabel(" ")</code>     | "not entered"  |
| <code>stockLabel("abc")</code>   | "not a number" |
| <code>stockLabel(null)</code>    | "not entered"  |
| <code>copies    'unknown'</code> | "unknown"      |
| <code>copies ?? 'unknown'</code> | 0              |
| <code>null == undefined</code>   | true           |

**Compare** — `live-code/01-coercion-and-truth/checkpoint.png`. Fourteen rows · `stockLabel("")` and `stockLabel(" ")` say "not entered", not "sold out" · `stockLabel(null)` prints a row and the console is clean · `copies ?? 'unknown'` is 0.

CYCLE 2 · 25 MIN

# THE ARRAY YOU WERE HANDED

A function that sorts the list it receives — without touching it. `sort` reorders in place **and** returns the array, so it looks safe; `[...list]` before every `sort`; a comparator returns a number; a missing year goes last with `?? Infinity`; Hebrew and English sort with `localeCompare(..., 'he')`.

## 1 I DO

10 MIN

3 on the next four slides, 7 in the editor · live-code/02-the-array-you-were-handed · `list.sort(...)` (wrong) · `[...list]` · `null - 1815` · `?? Infinity` · `[...names].sort()` vs `localeCompare`

## 2 WE DO

5 MIN

you type steps 2–4 · compare with checkpoint 2 · the first row and the last row are the same

## 3 YOU DO

8 MIN

`newestFirst(list)` — newest first, the book with no year still last · AI as partner: ask why, then print it

## 4 DEBRIEF

2 MIN

the function returned the right answer — what was wrong? · ↗ reference

YOU DO · AI

➤ REFERENCE · 14, 15, 17–21

# SIX PROGRAMS, AND THE ENGINE ANSWERS

identical steps, one difference – and an `a === b` asked after each

⌕ reload open ↗

SIX PROGRAMS, STEPPED, WITH THE ENGINE ANSWERING

Program 1 · a number is copied ▼ Step Reset step 0 of 3

```
let a = 10;  
let b = a;  
b = 99;
```

Press Step.

BINDINGS, AFTER THIS STEP

| NAME | VALUE     |
|------|-----------|
| a    | undefined |
| b    | undefined |
|      |           |

► **STOP PROGRAM 4 ON ITS LAST STEP.** Ask who thinks `prices` is unchanged. Count the hands. Then press Step.

# THE FUNCTION THAT TOLD THE TRUTH AND LIED

## WHAT IT SAYS

```
1 function addTax(list) {  
2   list.sort((a, b) => a - b);  
3   return list.map((p) => p * 1.17);  
4 }
```

Returns a new array. **And reorders yours.**

## WHAT IT SHOULD SAY

```
1 function addTax(list) {  
2   const copy = [...list];  
3   copy.sort((a, b) => a - b);  
4   return copy.map((p) => p * 1.17);  
5 }
```

One line. Now the name is honest.

**sort** and **reverse** mutate **and**  
**return** the array, so they look safe.

## const IS ABOUT THE NAME

```
1 const list = [1, 2];
2 list.push(3);           // fine. the array was never frozen.
3
4 const total = 0;
5 total = 1;              // TypeError: Assignment to constant variable.
6
7 // use const everywhere. use let when the NAME must point somewhere else.
```

### THE HABIT

`const` by default. `let` when you reassign. `var` never — the reference deck says why, and the assignment checks it.

### NOT A PROMISE OF IMMUTABILITY

If you actually need that, copy: `[...list]`, `{ ...item }`. Both are shallow.

# COMPARATORS, AND THE ONLY TWO YOU NEED

## WRONG

```
1 list.sort()
2 list.sort((a, b) => a.title > b.title)
```

The first compares strings. The second returns `true / false`, and `sort` wants a **number**.

## RIGHT

```
1 [...list].sort((a, b) => a.year - b.year)
2 [...list].sort((a, b) =>
3   a.title.localeCompare(b.title, 'he'))
```

Negative, zero, positive. And a copy, so the caller keeps its order.

## A MISSING VALUE GOES LAST

```
1 (a, b) => (a.year ?? Infinity)
2           - (b.year ?? Infinity)
```

`null - 1815` is `-1815`: without `??` the book with no year comes **first**. `?? 0` does the same. Every real year is smaller than `Infinity`.

## TIES ARE NOT FREE

```
1 b.rating - a.rating
2 || a.title.localeCompare(b.title, 'he')
```

## CHECKPOINT 2

## YOUR SCREEN SHOULD LOOK LIKE THIS

Week 8 · cycle 2 · the array you were handed

## The list you were handed

One row for every print() in app.js, in the order they ran.

| what app.js printed | the value                                                                                 |
|---------------------|-------------------------------------------------------------------------------------------|
| shelf, as handed    | ["The Hobbit (1937)", "{(1968) םל ןכאל", "dune (1965)", "Field notes (?)", "Emma (1815)"] |
| null - 1815         | -1815                                                                                     |
| byYear(shelf)       | ["Emma (1815)", "The Hobbit (1937)", "dune (1965)", "{(1968) םל ןכאל", "Field notes (?)"] |
| [...names].sort()   | ["Emma", "Field notes", "The Hobbit", "dune", "םל ןכאל םל"]                               |
| byTitle(shelf)      | ["{(1968) םל ןכאל", "dune (1965)", "Emma (1815)", "Field notes (?)", "The Hobbit (1937)"] |
| shelf, afterwards   | ["The Hobbit (1937)", "{(1968) םל ןכאל", "dune (1965)", "Field notes (?)", "Emma (1815)"] |

**Compare** — `live-code/02-the-array-you-were-handed/checkpoint.png` . The first row and the last row are identical · `Field notes (?)` is last in `byYear` · `byTitle` starts with the Hebrew title.

CYCLE 3 · 30 MIN

# THIS AND ARROWS: WHOSE `this` IS IT?

Functions are values — you hand them over by name, without parentheses. An arrow with braces needs a `return`. And `this` is decided where the function is **called**, not where it was written. The rule that comes out of it: **arrow for callbacks, function for methods**.

## 1 I DO

10 MIN

3 on the next three slides, 7 in the editor · live-code/03-this-and-arrows · `(book) => { book.title }` (wrong) · `map(toTitle)` vs `map(toTitle())` · `count()` and `count`, detached · `tagged()` with an arrow inside

## 2 WE DO

8 MIN

you type steps 2–4 · `break` tagged() once with function (`book`), then put the arrow back · compare with checkpoint 3

## 3 YOU DO

10 MIN

lab/warmup/ — five functions, four wrong, the referee says "five of five"

## 4 DEBRIEF

2 MIN

"arrows fix this, so everything is an arrow" breaks the method · ↗ reference

YOU DO · AI-FREE

➤ REFERENCE · 24, 25

## ARROW OR FUNCTION – AND IT IS NOT ABOUT BREVITY

### SAME

```
1 const a = (x) => x * 2;  
2 function b(x) { return x * 2; }
```

### THE TRAP

```
1 const f = (x) => { x * 2 };  
2 f(3); // undefined
```

Braces make a **body**. A body needs `return`.

### EITHER OF THESE

```
1 const f = (x) => x * 2;  
2 const g = (x) => {  
3   return x * 2;  
4 };
```

Returning an object needs the parentheses:

`(item) => ({ id: item.id })`.

Without them the braces are a body again.

# ONE FUNCTION, FIVE CALL SITES

nothing is redefined between the rows – only the call changes

↺ reload

open ↗

ONE FUNCTION OBJECT, FIVE CALL SITES

```
const owner = { name: 'the shelf' };

function probe() {
  return this?.name ?? '(no this)';
}

owner.probe = probe;           // the SAME function, given a second name
```

Run all five

☐ define `probe` as an arrow instead

| HOW IT IS CALLED                                | THIS?.NAME                | WHY                                                                                                                                                                         |
|-------------------------------------------------|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>probe()</code>                            | <b>(no this)</b>          | Nothing before the dot, because there is no dot. In a module (and in strict mode) <code>'this'</code> is <code>'undefined'</code> rather than the window.                   |
| <code>owner.probe()</code>                      | <b>the shelf</b>          | The object before the dot becomes <code>'this'</code> . This is the rule people think is the only rule.                                                                     |
| <code>probe.call(other)</code>                  | <b>a different object</b> | <code>'call'</code> , <code>'apply'</code> and <code>'bind'</code> say what <code>'this'</code> is, out loud. This is the escape hatch.                                     |
| <code>[1].map(owner.probe)[0]</code>            | <b>(no this)</b>          | The function was passed, not called through the object. <code>'map'</code> calls it with no object in front, so the object is gone. This is the bug you will hit in week 9. |
| <code>[1].map(() =&gt; owner.probe())[0]</code> | <b>the shelf</b>          | The arrow is the callback; the call inside it is still a method call through <code>'owner'</code> . The fix is not "use an arrow" — it is "keep the dot".                   |

`probe === owner.probe` is true. There is one function. It does not belong to `'owner'`, it is not a copy, and nothing about it changed between the rows above — only the call site did.

► **READ THE FIVE CALL SITES ALOUD AND VOTE ON EACH, BEFORE REVEALING.** The method row is unanimous. The callback row is not.

## FOUR RULES, CHECKED IN THIS ORDER

- 1 new**  
a brand-new object. Not used in this course.
- 2 call / apply / bind**  
whatever you said, explicitly.
- 3 a dot**  
`owner.probe()` — the thing before the dot.
- 4 none of the above**  
`undefined` in a module. Not the window.

### AND OUTSIDE ALL FOUR

An **arrow** has no `this` of its own, so none of the rules apply. It uses the `this` of wherever it was written.

### THE PRACTICAL RULE

**Arrow for callbacks. Function for methods.** That is the whole decision, and it is what the demo just showed.

## CHECKPOINT 3

## YOUR SCREEN SHOULD LOOK LIKE THIS

Week 8 · cycle 3 · this and arrows

## Whose this is it?

One row for every print() or attempt() in app.js, in the order they ran.

| what app.js printed        | the value                                                                      |
|----------------------------|--------------------------------------------------------------------------------|
| shelf.books.map(toTitle)   | ["The Hobbit", "dune", "היכל של"]                                              |
| shelf.books.map(toTitle()) | TypeError: Cannot read properties of undefined (reading 'title')               |
| shelf.count()              | "Reading list: 3 books"                                                        |
| count(), no dot            | TypeError: Cannot read properties of undefined (reading 'label')               |
| shelf.tagged()             | ["Reading list · The Hobbit", "Reading list · dune", "Reading list · היכל של"] |

**Compare** — `live-code/03-this-and-arrows/checkpoint.png`. Five rows, two of them red **on purpose**: `map(toTitle())` and `count(), no dot`. They are the proof of the two rules, and they stay.

BREAK

# 10 MINUTES

Back at 100. Cycles 4 and 5 are the AI cycles — a chat window open (free or paid), `project/templates/FEATURE_SPEC.md` and `PROMPTS.md` where you can copy them, and a copy of `ai-lab/04-spec-then-generate/artifact/` as your practice folder.

CYCLE 4 · 25 MIN

## A SPEC FIRST, THEN CLAUDE WRITES

You copy the template into `specs/filter-and-sort.md` and fill it — \$2 says `books` is never reordered. Claude implements it. The answer reads well, line by line, and the bench finds two red rows: the right list returned, **your** list reordered. Then you write the prompt again, with the rule in it.

### 1 I DO

8 MIN

3 on the next three slides, 5 in the editor and the chat · ai-lab/04-spec-then-generate · the good spec on screen · specs/filter-and-sort.md typed · P1 live

### 2 WE DO

6 MIN

your own specs/filter-and-sort.md from the template · P1 in your chat · bench.html · find the red row before it gets a name

### 3 YOU DO

9 MIN

your own P2 with the rule in the prompt · 9 of 9 · search the file for document, \$, var, export

### 4 DEBRIEF

2 MIN

a rule in the spec is not a rule that is kept · ↗ reference

YOU DO · AI

# THE LOOP

## Claude chat

- 1 to think. Ask for options.

**No code yet.**

## FEATURE\_SPEC.md

- 2 to decide. Written down is checkable.

## Claude Code

- 3 to implement — from the spec, not from a sentence.

## Review

- 4 against your own criteria. `git diff`, not memory.

## Iterate the spec

- 5, not the output. **Nobody does step 5.** It is the one that compounds.

# THE SAME FEATURE, FROM TWO REQUESTS

both implementations are real, and both are run against the same eight cases

↺ reload

open ↗

## ONE FEATURE, TWO REQUESTS, EIGHT CASES

☐ only run the happy path Click a row to see which line of the spec it comes from.

### THE REQUEST MOST PEOPLE SEND

"Add a search box that filters the list."

1. — nothing else was said —

```
export function filterItems(items, query) {  
  return items.filter((item) =>  
    item.title.includes(query)  
  );  
}
```

fails 4 of 8

### THE SAME FEATURE, SPECIFIED

"Implement `filterItems` per the spec below."

1. Section 2 — `query` is a string on state; it starts as `''`.
2. Section 5 — an empty query shows everything, not nothing.
3. Section 5 — leading and trailing spaces are trimmed before matching.
4. Section 5 — matching is case-insensitive.
5. Section 5 — an item whose title is missing must not crash the list.
6. Section 6 — with no matches the list is empty and the counter reads 0.

```
export function filterItems(items, query) {  
  const term = query.trim().toLowerCase();  
  if (term === '') return items;  
  return items.filter((item) =>  
    (item.title ?? '').toLowerCase().includes(term)  
  );  
}
```

passes all 8

► **TICK "ONLY RUN THE HAPPY PATH" FIRST.** Both green, both finished-looking. Then untick it, and say nothing.

# WHAT A SPEC HAS THAT A SENTENCE DOES NOT

- §1 What the user can do after, and could not before. One sentence. If it contains "and", it is two features.
- §2 What changes in `state` — before and after.  
**This week that is your data model.**
- §5 Edge cases, at least three. **AI writes the happy path very well and the edges almost never.**
- §6 How you know it works — things you can **see**, not "the function returns an array".
- §7 The review, written **after** the code exists.  
This is the graded part.

## YOU ALREADY HAVE THE FILE

`project/templates/FEATURE_SPEC.md` . Copy it once per feature into `specs/` — today, `specs/filter-and-sort.md` .

## THIS WEEK ONLY

§4 (the events) says “**not yet — week 9**”, in Hebrew in your file. There is no behaviour yet, and pretending otherwise is worse than an empty section.

CYCLE 4 · ON SCREEN BEFORE ANYONE WRITES

# WHAT A SPEC THAT WORKS LOOKS LIKE

## §2 · WHAT IS NOT ADDED

`visibleShelf` is **not** stored — the list on screen is derived. That line is what Claude will try to add, so it is written down before it asks.

## §5 · CASES THAT PASS THE DEMO AND FAIL A USER

Spaces only · no match · a year that is missing, **last, not year 0** · Hebrew and English together. Every red row on a bench is a line here, or should have been.

## §7 · WRITTEN AFTER THE CODE

What came back, what was wrong, what I did. It is the graded part, and it is empty until cycle 5.

► **AI-EXEMPLARS/EXEMPLARS/FEATURE-SPEC-GOOD.MD** in your course materials — the short version of the project example's real spec, `project/templates/examples/FEATURE_SPEC.md` . Read it for §2, §5 and §7.

## CHECKPOINT 4 · THREE QUESTIONS

# BEFORE YOU LEAVE CYCLE 4

- 1 Did you find a row where the returned list is right and your list was reordered?**  
And read it back to \$2 of the spec, before anybody named it.
- 2 Is the rule in the body of your P2, and does what came back pass 9 of 9?**  
“Copy before any sort, on every path, including the empty query.”
- 3 Did you search the P1 file for what you did not ask for?**  
`wireUp` and `$` — and `var` , found zero times. Write down the search that found nothing, too.

CYCLE 5 · 20 MIN

## THE REVIEW, THEN THE JOURNAL

Claude is asked to review its own file against the spec. Seven rows, seven ticks — and the tick on \$2 repeats the sentence that wrote the bug. The bench says the opposite. Then you write the two documents that are graded: \$7 of the spec, and entry 1 of **PROMPTS.md** — from what ran, not from what was said.

### 1 I DO

7 MIN

2 on the next slide, 5 in the chat ·  
ai-lab/05-review-then-journal · P3:  
"review your file" · the bench  
beside it · P4: the red row as  
evidence

### 2 WE DO

5 MIN

the good review on screen, then  
the bad one · \$7 of specs/filter-and-  
sort.md, three rows, together

### 3 YOU DO

6 MIN

the good journal entry on screen,  
then the bad one · entry 1 of your  
project/PROMPTS.md

### 4 DEBRIEF

2 MIN

a review by the author is a claim;  
the proof is what ran · ↗ reference

YOU DO · AI

# ONE WINDOW, ONE FEATURE, ONE REVIEW

## 1 Left

the chat, or Claude Code — the file it wrote, and its own review of it.

## 2 Middle

`specs/filter-and-sort.md`, §7 still empty.

## 3 Right

the bench, `bench.html` — what ran, row by row.

## 4 Below

`git diff`: the thing nobody asked for (`wireUp`, `$`).

## 5 Then write

§7 of the spec, and entry 1 of `PROMPTS.md` — from the bench, not from the review.

► **A REVIEW BY THE AUTHOR IS A CLAIM.** The bench is the proof. **NO SUBSCRIPTION?** Paste the file and the spec into the free chat, then the red row — the review you write is identical, and it is the part worth points.

CYCLE 5 · ON SCREEN BEFORE PART 3 IS GRADED

# A REVIEW OF GENERATED CODE: GOOD, THEN BAD

## AI-REVIEW-JS-GOOD.MD

**Which one** — `byDecade`. **The claim, and the part of it that is true** — `Math.floor` is right, `=== undefined` does skip a missing field. **What happens** — `year: null` lands in decade 0. **The proof** — three lines and what they printed:  
`{"0": ["Debut"], "2000": ["Kid A"]}`. **The fix** — `== null`, one change.

## AI-REVIEW-JS-BAD.MD

Names the wrong function (“a Set is a hash, the order is random” — it keeps insertion order). “The explanation lied.” “I read it and understood.” Rewrote all three. **True in places, and worth nothing.**

► **AI-EXEMPLARS/EXEMPLARS/AI-REVIEW-JS-GOOD.MD** and **-BAD.MD** in your course materials — the same six sections as part 3 of the assignment, on three different functions.

CYCLE 5 · ON SCREEN BEFORE YOU WRITE ENTRY 1

## A JOURNAL ENTRY: GOOD, THEN BAD

### PROMPT-JOURNAL-GOOD.MD

**The exact prompt.** What came back. **What was wrong, and how I checked** — a click, a number, a bench row. What I did. And the line that turns one entry into a rule: **what I learned about the prompt itself** — “quote the rule in the body of the prompt”.

### PROMPT-JOURNAL-BAD.MD

“I asked Claude to build the filter and sort.” “There were a few small things; I fixed them.” “Claude Code saves a lot of time.” **Every word is true, and it is worth nothing** — to the grader, and to you in week 13.

► **AI-EXEMPLARS/EXEMPLARS/PROMPT-JOURNAL-GOOD.MD** and **-BAD.MD**. Your entry 1 goes in `project/PROMPTS.md` (copy `project/templates/PROMPTS.md` if you have none) and it is about cycle 4: P1, the red row, P2, and what you learned about the prompt.

## CHECKPOINT 5 · THREE QUESTIONS

## BEFORE YOU LEAVE CYCLE 5

- 1 Does §7 of your spec quote the bench row, and not the tick from P3?**  
The input, and the word REORDERED.
- 2 In the §2 row, did you write which part of the explanation is true, and why the conclusion is not?**  
`filter` does return a new array. On the empty path there is no `filter`.
- 3 Is the last line of your journal entry a rule for the next prompt?**  
Not “Claude is really good”.

ASSIGNMENT · 145-175

# CLASS ASSIGNMENT 8: ASSEMBLE, THEN EXTEND

## 1 Predict · cycles 1-3

`predict-he.md` — fourteen snippets: guess, run, then say why. The why is graded, not the guess.

## 2 Four pure functions · cycles 1-2

`filterByQuery`, `sortItems` (a copy · `?? Infinity` · `localeCompare`), `countByArtist`, `summarise.No var`.

## 3 Review what Claude wrote · cycle 5

`topRated`, `initials`, `formatRating`: one does not do what its explanation says. Name it, prove it by running it, fix only it.

## Spec, then pages · cycle 4

4 `FEATURE_SPEC.md` for your topic, then three pages: the toolbar *outside* `<ul id="list">`, `aria-current="page"` on the current link.

### PARTS 1-3 ARE YOURS

No AI in predict, the functions or the review. **Week 13 reads code the same way, offline.**

### PART 4 IS THE LOOP

Spec first, then Claude, then the review in §7. A spec written after the code scores zero there.

# THE MILESTONE, AND HOMEWORK 3

## THE PROJECT, BY THE END OF THIS WEEK

Three pages in HTML and Tailwind, invented data in the markup, a menu between them with `aria-current="page"`, the toolbar outside the list. **No JavaScript in the project yet** — week 9 wires it, week 10 draws it from state.

## HOMEWORK 3 — ASSIGNED TODAY

Seven drum pads and two dice, **built entirely by JavaScript from two arrays**: one listener for every pad, a history you can remove from, a form that fails well. It is DOM work — week 9 is the week that teaches it. **Due before lesson 10.**

## CLASS ASSIGNMENT 8

Moodle, repo URL + commit SHA, **before lesson 9**. The work sits in `week-08/`.

## THREE QUESTIONS

- 1 **Why is `'' == '0'` false**  
when `'' == 0` and `0 == '0'` are both true?
- 2 **A function returned the right sorted list.**  
What else do you check before you call it correct?
- 3 **"`filter` returns a new array, so your list is safe."**  
True sentence. When is the conclusion false?

If you can answer these three,  
you can read the code of week 9.  
If not, `live-code/*/end/` and  
the demos are yours.

# GO DEEPER

The reference deck for this week — every slide the lecture pointed at, in order.

[2 · Seven types, and one of them is a mistake](#) ← spine 5

[3 · Eight values are false. Everything else is not.](#) ← spine 5

[5 · ==, in the order the rules are applied](#) ← spine 5

[6 · Two ways to have nothing, and one to have a wrong number](#) ← spine 5

[7 · The two nothings, in arithmetic](#) ← spine 5

[8 · Converting on purpose](#) ← spine 5

[10 · Two operators that replace ten lines of guarding](#) ← spine 5

[12 · Modules: every file has a door](#) ← spine 5

[14 · You know this one. It is not the same here.](#) ← spine 10

[15 · Six methods, and you already know five of them](#) ← spine 10

[17 · The one that is not like the others](#) ← spine 10

[18 · Comparators](#) ← spine 10

[19 · toSorted\(\): the copy, built in](#) ← spine 10

[20 · sort is stable &mdash; and you still write the tie-breaker](#) ← spine 10

[21 · Taking things apart, and putting them together](#) ← spine 10

[23 · var, let, const](#) ← spine 13

[24 · Functions are values. That is not a slogan.](#) ← spine 16

[25 · this, by call site](#) ← spine 16

[27 · Three ways to declare, and they fail differently](#) ← spine 4

[28 · Hoisted is not the same as usable](#) ← spine 4

[29 · A closure is a function that remembers where it was born](#) ← spine 4

[30 · The same closure, in the file you will ship](#) ← spine 4

[31 · Live: build the store, in nine lines](#) ← spine 4

[32 · Where your code runs now](#) ← spine 4

[33 · What does this print?](#) ← spine 4

[34 · One stack, two queues, one rule](#) ← spine 4

[35 · The rule, in one sentence](#) ← spine 4

[36 · setTimeout\(fn, 0\) means "when I am free"](#) ← spine 4

[38 · From today, Claude writes code for you](#) ← spine 22

[39 · A spec you can check before you read any code](#) ← spine 22

[42 · The five that cost marks](#) ← spine 35

[43 · Where to look, in order](#) ← spine 35